

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №4
на тему
**«АСИММЕТРИЧНАЯ КРИПТОГРАФИЯ.
КРИПТОСИСТЕМА МАК-ЭЛИСА»**

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Цель работы	3
2	Ход работы	4
2.1	Генерация ключей	6
2.2	Зашифрование и расшифрование текста	7
2.3	Зашифрование и расшифрование файлов	8
2.4	Обнаружение искажений	9
	Заключение	11
	Приложение А (обязательное) Листинг программного кода	12
	Приложение Б (обязательное) Блок-схема алгоритма	33

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение принципов построения асимметричных криптосистем на примере криптосистемы Мак-Элиса, стойкость которой основана на сложности декодирования произвольного линейного кода. В работе используется современный вариант криптосистемы на квазициклических кодах с умеренной плотностью проверок (QC-MDPC), лежащий в основе схемы ВІКЕ. Задачи включают изучение математических основ криптосистемы – арифметики кольца двоичных многочленов по модулю $x^r - 1$, кодов QC-MDPC и итеративного декодера с переворотом битов, – проектирование и реализацию программного средства, осуществляющего генерацию ключевой пары, зашифрование открытым ключом и расшифрование закрытым ключом, а также тестирование разработанного программного средства.

Конечным результатом данной лабораторной работы должна быть разработанная программа, осуществляющая зашифрование и расшифрование данных криптосистемой Мак-Элиса как для текста, вводимого с клавиатуры, так и для файлов произвольного содержания.

2 ХОД РАБОТЫ

В ходе выполнения работы было создано программное средство на языке программирования *Rust*. Сторонние библиотеки для алгебры не используются, из внешних зависимостей подключена только библиотека `rand` для генерации случайных чисел.

Вся арифметика криптосистемы выполняется в кольце

$$R = \text{GF}(2)[x]/(x^r - 1). \quad (2.1)$$

Элемент R – многочлен степени меньше r с коэффициентами из $\text{GF}(2)$, то есть вектор из r бит: бит i является коэффициентом при x^i . Сложение элементов – поразрядное исключаящее ИЛИ. Поскольку $x^r = 1$, умножение на x^j – это циклический сдвиг вектора на j позиций в сторону старших индексов: бит, вышедший за позицию $r - 1$, возвращается в начало. Произведение многочленов при раскрытии скобок оказывается циклической свёрткой их векторов коэффициентов,

$$(ab)_k = \sum_{i+j \equiv k \pmod{r}} a_i b_j, \quad (2.2)$$

и вычисляется как сумма сдвигов одного множителя на позиции единиц другого: $ab = \sum_{i \in \text{supp}(a)} x^i b$. Если множитель разрежен и содержит d единиц, произведение требует всего d сдвигов.

Векторы хранятся в двух формах. Плотная форма – массив 64-битных машинных слов, в котором бит i находится в слове $\lfloor i/64 \rfloor$ на позиции $i \bmod 64$, а биты за позицией $r - 1$ в последнем слове всегда нулевые. Сложение и сдвиг в такой форме обрабатывают по 64 коэффициента за операцию, причём сдвиг складывается из сдвига массива влево на j бит и вправо на $r - j$ бит, возвращающего вышедшие биты в начало. Разреженная форма – упорядоченный список позиций единиц; в ней хранятся секретный ключ и вектор ошибок, а умножение разреженного вектора на плотный выполняется ровно за столько сдвигов, сколько в нём единиц.

Модуль r выбирается простым, причём так, чтобы 2 было первообразным корнем по модулю r . Тогда $x^r - 1 = (x + 1)\Phi_r(x)$, где многочлен $\Phi_r(x) = 1 + x + \dots + x^{r-1}$ неприводим, и элемент R обратим тогда и только тогда, когда его вес нечётен и он не равен вектору из одних единиц. Кольцо при этом не является полем: например, $(1 + x)\Phi_r(x) = x^r - 1 = 0$. Обратный элемент вычисляется расширенным алгоритмом Евклида для пары $(x^r - 1, a)$ с инвариантом $r_i \equiv v_i a \pmod{x^r - 1}$: когда остаток r_i становится равен 1, соответствующий v_i и есть a^{-1} . Остатки хранятся как обычные многочлены из $r + 1$ коэффициента, так как $x^r - 1$ имеет степень ровно r , а коэффициенты v_i – сразу как элементы R . Над $\text{GF}(2)$ каждый шаг деления сводится к прибавлению сдвинутого делителя. Возведение в квадрат в характеристике 2 – перестановка битов, $a^2 = \sum a_i x^{2i \bmod r}$.

Код QC-MDPC задаётся двумя разреженными многочленами $h_0, h_1 \in R$, в каждом из которых ровно d единиц. Код длины $n = 2r$ состоит из пар (u, v) , для которых $uh_0 + vh_1 = 0$; его проверочная матрица $H = [\text{circ}(h_0) \mid \text{circ}(h_1)]$ составлена из двух циркулянтных блоков $r \times r$, каждая строка которых – циклический сдвиг предыдущей. Поэтому матрица задаётся двумя строками, а умножение на неё сводится к произведению многочленов. Каждая проверка охватывает $w = 2d$ бит, и при $w \approx \sqrt{n}$ проверочная матрица имеет умеренную плотность: разреженность нужна декодеру, а найти слово такого малого веса в открытом коде противник не может.

Ключевая пара строится следующим образом. Выбираются случайные h_0 и h_1 с d единицами каждый, при нечётном d многочлен h_1 обратим, и вычисляется $q = h_0 h_1^{-1}$. Все кодовые слова имеют вид (u, uq) , так что q задаёт код, но разреженные h_0 и h_1 по плотному q не восстанавливаются. Чтобы сообщение не проступало в шифротексте, вводится случайный обратимый плотный многочлен a нечётного веса, аналог матрицы S классической схемы. Открытым ключом являются $(A, B) = (a, aq)$ и число t , закрытым – h_0, h_1 и a^{-1} . В программе используются параметры ВКЕ уровня 1, рассчитанные на стойкость около 128 бит (таблица 2.1).

Таблица 2.1 – Параметры криптосистемы (ВКЕ, уровень 1)

r	d	$w = 2d$	t	Открытый ключ, байт	Текст в блоке, байт
12 323	71	142	134	3082	1540

Зашифрование блока $m \in R$ выполняется по формулам

$$c_0 = mA + e_0, \quad c_1 = mB + e_1, \quad (2.3)$$

где пара (e_0, e_1) содержит в сумме ровно t единиц. Для этого из $2r$ позиций выбираются t различных случайных: позиции меньше r образуют e_0 , остальные после вычитания $r - e_1$. Пара (ma, maq) является кодовым словом, поэтому шифротекст отличается от кодового слова ровно в t позициях. Благодаря случайному вектору ошибок зашифрование вероятностное.

Расшифрование начинается с вычисления синдрома

$$s = c_0 h_0 + c_1 h_1 = mah_0 + mah_0 h_1^{-1} h_1 + e_0 h_0 + e_1 h_1 = e_0 h_0 + e_1 h_1, \quad (2.4)$$

который зависит только от ошибок: слагаемые с сообщением взаимно уничтожаются. Ошибку находит итеративный декодер с переворотом битов. Ошибка в бите j части e_0 добавляет к синдрому многочлен $x^j h_0$, то есть бит j участвует в d проверках с номерами $(j + p) \bmod r$, $p \in \text{supp}(h_0)$. Счётчик бита – число невыполненных проверок среди них:

$$\text{upc}_0(j) = |\text{supp}(s) \cap \{(j + p) \bmod r : p \in \text{supp}(h_0)\}|, \quad (2.5)$$

и аналогично $urs_1(j)$ с h_1 . Ошибочный бит сам портит свои проверки, поэтому его счётчик в среднем больше: для принятых параметров около 43 из 71 против 28 у верного бита. На каждой итерации все $2r$ счётчиков вычисляются по одному синдрому, после чего переворачиваются биты, счётчик которых не меньше порога

$$T = \max(\lfloor 0.0069722 \cdot |s| + 13.530 \rfloor, 36), \quad (2.6)$$

взятого из спецификации VIKЕ. Порог зависит от веса синдрома: вначале он высокий и переворачиваются только явные ошибки, а по мере исправления опускается к 36. Переворот бита j части e_0 сопровождается прибавлением к синдрому $x^j h_0$, то есть переворотом тех же d позиций, по которым считался счётчик. Декодер останавливается, когда синдром обнуляется; обычно на это уходит 3–4 итерации. Декодер вероятностный и при неудачном расположении ошибок может не сойтись, поэтому число итераций ограничено десятью. Отказом считаются исчерпание итераций, итерация без единого переворота и найденная ошибка, вес которой не равен t .

По найденной ошибке вычисляется $c_0 + e_0 = ta$, и сообщение восстанавливается умножением на a^{-1} . Дополнительно проверяется, что $c_1 + e_1 = mB$, что отсекает ложные решения декодера. Блок-схема алгоритма приведена в приложении Б.

Сообщение разбивается на блоки по $k = \lfloor r/8 \rfloor = 1540$ байт: блок занимает младшие $8k$ бит элемента R , а старшие $r \bmod 8 = 3$ бита остаются нулевыми. Порядок бит – младший бит каждого байта первым. К сообщению всегда дописывается байт 01 и нулевые байты до длины, кратной k , поэтому при расшифровании дополнение снимается однозначно: отбрасываются нулевые байты в конце и один байт 01. Каждая половина шифроблока записывается в $\lfloor r/8 \rfloor = 1541$ байт, и шифроблок занимает 3082 байта на каждые 1540 байт сообщения, то есть шифротекст примерно вдвое длиннее открытого текста.

Перед расшифрованием проверяется, что длина шифротекста кратна длине шифроблока. Если какой-либо блок не декодируется, не проходит контроль $c_1 + e_1 = mB$ или дополнение оказывается некорректным, программа сообщает об ошибке расшифрования.

Исходный код арифметики кольца приведён в приложении А в листинге А.1, криптосистемы – в листинге А.2.

2.1 Генерация ключей

При запуске программа генерирует ключевую пару. Выбираются 71 случайная позиция для h_0 и столько же для h_1 , h_1 обращается алгоритмом Евклида, и вычисляется $q = h_0 h_1^{-1}$ умножением разреженного вектора на плотный. Затем выбирается случайный многочлен a нечётного веса, не равный вектору из одних единиц, вычисляются a^{-1} и $B = aq$. Вся генерация заняла около 8 мс.

Меню программы и сведения о ключах представлены на рисунке 2.1. Открытый ключ состоит из параметров r , t и двух многочленов A и B по 1541

байт, всего 3082 байта. Оба многочлена плотные, их вес близок к $r/2 \approx 6160$, поэтому программа выводит только первые 32 байта и вес. Закрытый ключ – это позиции 71 единицы в h_0 и в h_1 , из которых выводятся первые 64, и плотный a^{-1} . Разреженные h_0 и h_1 , заданные всего 71 позицией каждый, дают плотные q и B : структура кода в открытом ключе не видна. Новую ключевую пару можно сгенерировать командой `g`.

```
Generating keys for r = 12323, d = 71, t = 134...
Done in 7.61ms

McEliece cryptosystem (QC-MDPC) – input/output: text, r = 12323, t = 134
1) encrypt      2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): k

Public key (3082 bytes)
  r, t      : 12323, 134
  A = a     : B90C0774C7D4C71C34B6FDD21959613CF8BD1789C04AFAEB3D305773F6C74F6A... (weight 6185, 1541 bytes)
  B = a·q   : E36213E41EEE5F95D91EEF082423BE54C8F127C289B3D89927F914AA01AF2450... (weight 6255, 1541 bytes)
Private key
  supp(h0)  : 603, 614, 645, 719, 863, 1023, 1094, 1152, 1767, 1940, 2126, 2276, 2495, 2507, 2734, 3558,
              3727, 3873, 3950, 4021, 4067, 4133, 4151, 4391, 4674, 4737, 5346, 5604, 5616, 5836, 5889, 6167, 6390, 6471,
              6596, 6624, 6745, 6805, 7139, 7253, 7544, 7583, 7731, 7862, 7991, 8094, 8272, 8282, 8327, 8400, 8459, 8663,
              8756, 8888, 9151, 9171, 9303, 9325, 9766, 10017, 10121, 10407, 10699, 10726, ... (7 more)
  supp(h1)  : 143, 171, 691, 1009, 1098, 1115, 1342, 1414, 1792, 2117, 2213, 2353, 2405, 2422, 2519, 2609,
              2763, 2852, 2880, 3075, 3234, 3481, 3507, 3862, 4066, 4112, 4225, 4354, 4363, 4840, 5138, 6042, 6129, 6351,
              6825, 6985, 7075, 7142, 7288, 7364, 7440, 7844, 8039, 8100, 8104, 8209, 8242, 8456, 8499, 8541, 8695, 8806,
              8904, 9188, 9243, 9658, 9676, 10296, 10398, 10403, 10708, 10722, 10979, 11000, ... (7 more)
  aP-1     : 473B50146FC4800A2C0A5D5FF9C9CF300F27D4306DF31ACC4347666FFAD7F579... (weight 6117, 1541 bytes)
```

Рисунок 2.1 – Меню программы и ключевая пара

2.2 Зашифрование и расшифрование текста

Результат работы программы для текста, вводимого с клавиатуры, представлен на рисунке 2.2. Шифротекст выводится полностью, в 6164 шестнадцатеричных цифрах, поэтому на рисунке середина каждого шифротекста опущена. Сообщение «Привет, QC-MDPC!» в кодировке UTF-8 занимает 22 байта. Вместе с байтом дополнения и 1517 нулевыми байтами оно образует один блок из 1540 байт, который зашифровывается в шифроблок из 3082 байт.

Повторное зашифрование того же сообщения тем же ключом даёт другой шифротекст, так как каждый раз выбирается новый вектор ошибок. При этом два шифротекста почти совпадают: их сумма равна $(c_0 + c'_0, c_1 + c'_1) = (e_0 + e'_0, e_1 + e'_1)$ и содержит не больше $2t = 268$ единиц. В показанном примере шифротексты различаются в 266 битах: одна позиция ошибки совпала в обоих векторах и при сложении сократилась. Различия затрагивают 257 байт из 3082. Поэтому одно и то же сообщение в базовой схеме нельзя шифровать повторно: сумма шифротекстов раскрывает позиции ошибок.

При расшифровании шифротекст вводится в шестнадцатеричном виде. Программа проверяет его длину, вычисляет синдром, находит 134 ошибки декодером, умножает $c_0 + e_0$ на a^{-1} , проверяет $c_1 + e_1 = mB$ и снимает дополнение, после чего выводит исходный текст и его длину. Расшифрование блока

заняло около 10 мс против менее чем 0,1 мс на зашифрование: основное время уходит на декодер.

```
McEliece cryptosystem (QC-MDPC) - input/output: text, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 1

Plaintext: Привет, QC-MDPC!
mode      : encryption
time      : 54.80µs
plaintext : 22 bytes
encrypted : D8A767E75977247B3C16F58782CD94C0027BC6B96641B7362F6C307683C312CBBC8928B4D3D0E90482F7A24F722CC3B4F1
           A2D57E3EDFD0072DD54404E594D656C5D4F2BF14A10532E4855DF1D1C28A8D ... [опущено символов: 5964] ...
           68E05B79A21A109159089F5BE51ACE3E1CBFC507 (3082 bytes)

McEliece cryptosystem (QC-MDPC) - input/output: text, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 1

Plaintext: Привет, QC-MDPC!
mode      : encryption
time      : 72.40µs
plaintext : 22 bytes
encrypted : D8A667E75976247B8C16F58782CCB4C0027BC6B96641B7362F6C307683C312CBBC8928B4D3D0E90482F7A24F722CC3B4F1
           A2D57E3EDFD0072DD54404E594D656C5D4F2BF14A10532E4855DF1D1C28A8D ... [опущено символов: 5964] ...
           68E05B79A21A109159089F5BE51ACE3E1CBFC07 (3082 bytes)

McEliece cryptosystem (QC-MDPC) - input/output: text, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 2

Ciphertext (hex): D8A767E75977247B3C16F58782CD94C0027BC6B96641B7362F6C307683C312CBBC8928B4D3D0E90482F7A24F722CC3
                  B4F1A2D57E3EDFD0072DD54404E594D656C5D4F2BF14A10532E4855DF1D1C28A8D ... [опущено символов: 5964] ...
                  68E05B79A21A109159089F5BE51ACE3E1CBFC507
mode      : decryption
time      : 9.79ms
encrypted : 3082 bytes
decrypted : Привет, QC-MDPC! (22 bytes)
```

Рисунок 2.2 – Зашифрование и расшифрование текста (середина шифротекстов опущена)

2.3 Зашифрование и расшифрование файлов

В файловом режиме, включаемом командой `m`, программа читает файл целиком и записывает результат в файл, имя которого предлагается по умолчанию: при зашифровании добавляется расширение `.mce`, при расшифровании оно отбрасывается. Результат представлен на рисунке 2.3. Текстовый файл `plain.txt` размером 7470 байт вместе с дополнением образует 5 блоков и зашифровывается в файл из $5 \cdot 3082 = 15410$ байт. Расшифрованный файл `out.txt` побайтно совпадает с исходным. Зашифрование заняло 11 мс, расшифрование – 47 мс, около 9 мс на блок.

```

McEliece cryptosystem (QC-MDPC) - input/output: text, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): m

Switched to file mode

McEliece cryptosystem (QC-MDPC) - input/output: file, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 1

Plaintext file: plain.txt
Output file [plain.txt.mce]:
  mode      : encryption
  time      : 10.87ms
  input     : plain.txt (7470 bytes)
  output    : plain.txt.mce (15410 bytes)

McEliece cryptosystem (QC-MDPC) - input/output: file, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 2

Ciphertext file: plain.txt.mce
Output file [plain.txt]: out.txt
  mode      : decryption
  time      : 46.70ms
  input     : plain.txt.mce (15410 bytes)
  output    : out.txt (7470 bytes)

McEliece cryptosystem (QC-MDPC) - input/output: file, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): m

Switched to text mode

```

Рисунок 2.3 – Зашифрование и расшифрование файла

2.4 Обнаружение искажений

Реакция программы на некорректный шифротекст показана на рисунке 2.4. В качестве исходного взят первый шифротекст из рисунка 2.2. В первом случае в нём изменён младший бит 21-го байта, 66 заменено на 67. Если этот бит не совпадает с позицией ошибки, ошибок становится $t + 1$, иначе $t - 1$. В обоих случаях декодер либо не сходится, либо находит вектор ошибок веса, отличного от t , и блок отвергается. В отличие от декодера кодов Гоппы, исправляющего любое число ошибок до t , здесь проверяется вес ровно t , поэтому изменение одного бита обнаруживается независимо от того, попал ли он на позицию ошибки.

Во втором случае шифротекст укорочен на один байт до 3081 байта. Он отвергается ещё до расшифрования, поскольку его длина не кратна длине шифроблока. В третьем случае после генерации новой ключевой пары командой `g` исходный шифротекст расшифровывается уже чужим закрытым ключом: син-

дром относительно другого кода не соответствует никакому вектору ошибок веса t , и декодер сообщает об отказе.

```
McEliece cryptosystem (QC-MDPC) – input/output: text, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 2

Ciphertext (hex): D8A767E75977247B3C16F58782CD94C0027BC6B966741B7362F6C307683C312CBBC8928B4D3D0E90482F7A24F722CC3
B4F1A2D57E3EDFD0072DD54404E594D656C5D4F2BF14A10532E4855DF1D1C28A8D ... [опущено символов: 5964] ...
68E05B79A21A109159089F5BE51ACE3E1CBFC507
Error: a block failed to decrypt: decoding failure, wrong key or damaged ciphertext

McEliece cryptosystem (QC-MDPC) – input/output: text, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 2

Ciphertext (hex): D8A767E75977247B3C16F58782CD94C0027BC6B96641B7362F6C307683C312CBBC8928B4D3D0E90482F7A24F722CC3
B4F1A2D57E3EDFD0072DD54404E594D656C5D4F2BF14A10532E4855DF1D1C28A8D ... [опущено символов: 5962] ...
7468E05B79A21A109159089F5BE51ACE3E1CBFC5
Error: a block failed to decrypt: decoding failure, wrong key or damaged ciphertext

McEliece cryptosystem (QC-MDPC) – input/output: text, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): g

Generating keys for r = 12323, d = 71, t = 134...
Done in 7.04ms

McEliece cryptosystem (QC-MDPC) – input/output: text, r = 12323, t = 134
1) encrypt          2) decrypt
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 2

Ciphertext (hex): D8A767E75977247B3C16F58782CD94C0027BC6B96641B7362F6C307683C312CBBC8928B4D3D0E90482F7A24F722CC3
B4F1A2D57E3EDFD0072DD54404E594D656C5D4F2BF14A10532E4855DF1D1C28A8D ... [опущено символов: 5964] ...
68E05B79A21A109159089F5BE51ACE3E1CBFC507
Error: a block failed to decrypt: decoding failure, wrong key or damaged ciphertext
```

Рисунок 2.4 – Расшифрование искажённого шифротекста и расшифрование чужим ключом

Таким образом криптосистема Мак-Элиса на кодах QC-MDPC была изучена, реализована и протестирована в полном объёме как для текста, вводимого с клавиатуры, так и для файлов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены принципы построения асимметричных криптосистем на примере криптосистемы Мак-Элиса на квазициклических кодах с умеренной плотностью проверок. Было разработано программное средство на языке *Rust*, реализующее арифметику кольца $GF(2)[x]/(x^r - 1)$ над векторами, упакованными в машинные слова, обращение элементов кольца алгоритмом Евклида, итеративный декодер с переворотом битов и пороговой функцией ВКЕ, а на их основе – генерацию ключевой пары, зашифрование открытым ключом и расшифрование закрытым ключом над текстом, вводимым с клавиатуры, либо над файлами произвольного содержания.

Тестирование показало, что расшифрование восстанавливает исходное сообщение без искажений при любой длине, а искажение шифротекста, расшифрование чужим ключом и неверная длина обнаруживаются. Работа показала основные особенности криптосистемы. Её стойкость основана на NP-трудной задаче декодирования произвольного линейного кода, для которой неизвестны эффективные алгоритмы, в том числе квантовые, поэтому криптосистема относится к постквантовым. Квазициклическая структура позволяет хранить матрицу $r \times r$ одной строкой из r бит, благодаря чему открытый ключ занимает 3082 байта против 67 072 байт у кодов Гоппы с параметрами 1978 года, а все операции сводятся к сдвигам и исключаяющему ИЛИ.

Отдельно следует отметить отличия от классической схемы на кодах Гоппы. Декодер здесь не алгебраический, а итеративный и вероятностный: он изредка может не сойтись, и вероятность отказа резко растёт при завышенном числе ошибок, поэтому t подбирается вместе с r и d , а пороговая функция привязана к конкретному набору параметров. Статистика отказов, наблюдаемая противником, позволяет восстановить ключ (атака реакцией), поэтому на практике используется схема инкапсуляции ключа ВКЕ с одноразовыми ключами. Кроме того, базовая схема, как и классическая, податлива и не допускает повторного зашифрования одного сообщения.

Все задачи лабораторной работы выполнены в полном объёме.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Арифметика кольца $GF(2)[x]/(x^r - 1)$

```
use rand::Rng;
use rand::seq::index::sample;

#[derive(Clone, Debug, PartialEq, Eq)]
pub struct DenseBitVec {
    r: usize,
    words: Vec<u64>,
}

impl DenseBitVec {
    pub fn zero(r: usize) -> Self {
        let len = r.div_ceil(64);
        return Self {
            r,
            words: vec![0u64; len],
        };
    }

    pub fn one(r: usize) -> Self {
        let len = r.div_ceil(64);
        let mut words = vec![0u64; len];
        words[0] = 1;
        return Self { r, words };
    }

    fn mask(r: usize) -> u64 {
        return if r % 64 == 0 {
            u64::MAX
        } else {
            (1u64 << (r % 64)) - 1
        };
    }

    pub fn random_odd<R: Rng>(r: usize, rng: &mut R) -> Self {
        let len = r.div_ceil(64);
        let mask = Self::mask(r);
        loop {
            let mut words: Vec<u64> = (0..len).map(|_|
rng.next_u64()).collect();
            words[len - 1] &= mask;

            if words.iter().map(|w| w.count_ones()).sum::<u32>() % 2 == 0 {
                words[0] ^= 1;
            }

            if words.iter().map(|w| w.count_ones()).sum::<u32>() != r as u32
{
                return Self { r, words };
            }
        }
    }

    pub fn from_bytes(r: usize, bytes: &[u8]) -> Self {
        let len = r.div_ceil(64);
        let mask = Self::mask(r);
```

```

    assert!(r.div_ceil(8) >= bytes.len());

    let mut words: Vec<u64> = Vec::with_capacity(len);

    for block in bytes.chunks(8) {
        let mut block_bytes = [0u8; 8];
        block_bytes[..block.len()].copy_from_slice(block);
        words.push(u64::from_le_bytes(block_bytes));
    }

    words.resize(len, 0);
    words[len - 1] &= mask;

    return Self { r, words };
}

pub fn to_bytes(&self) -> Vec<u8> {
    let mut res: Vec<u8> = Vec::with_capacity(self.words.len() * 8);
    for word in &self.words {
        res.extend_from_slice(&word.to_le_bytes());
    }
    res.truncate(self.r.div_ceil(8));
    return res;
}

pub fn to_bit_string(&self) -> String {
    return (0..self.r)
        .map(|i| if self.get(i) { '1' } else { '0' })
        .collect();
}

pub fn r(&self) -> usize {
    return self.r;
}

pub fn get(&self, i: usize) -> bool {
    assert!(i < self.r, "bit {i} is out of range for r = {}", self.r);
    return (self.words[i / 64] >> (i % 64)) & 1 == 1;
}

pub fn set(&mut self, i: usize, value: bool) {
    assert!(i < self.r, "bit {i} is out of range for r = {}", self.r);
    if value {
        self.words[i / 64] |= 1u64 << (i % 64);
    } else {
        self.words[i / 64] &= !(1u64 << (i % 64));
    }
}

pub fn flip(&mut self, i: usize) {
    assert!(i < self.r, "bit {i} is out of range for r = {}", self.r);
    self.words[i / 64] ^= 1u64 << (i % 64);
}

pub fn weight(&self) -> usize {
    return self.words.iter().map(|w| w.count_ones() as usize).sum();
}

pub fn is_zero(&self) -> bool {
    return self.words.iter().all(|&w| w == 0);
}

pub fn support(&self) -> Vec<usize> {

```

```

    let mut res = Vec::new();
    for (k, &word) in self.words.iter().enumerate() {
        let mut w = word;
        while w != 0 {
            res.push(k * 64 + w.trailing_zeros() as usize);
            w &= w - 1;
        }
    }
    return res;
}

pub fn xor(&self, other: &DenseBitVec) -> DenseBitVec {
    let mut res = self.clone();
    res.xor_assign(other);
    return res;
}

pub fn xor_assign(&mut self, other: &DenseBitVec) {
    assert_eq!(self.r, other.r, "vectors from different rings");
    for (a, b) in self.words.iter_mut().zip(&other.words) {
        *a ^= b;
    }
}

pub fn xor_sparse_assign(&mut self, other: &SparseBitVec) {
    assert_eq!(self.r, other.r, "vectors from different rings");
    for &i in &other.set_bits {
        self.flip(i);
    }
}

fn xor_shifted(&mut self, src: &DenseBitVec, j: usize) {
    let r = self.r;
    let n = self.words.len();
    let j = j % r;

    let (q, s) = (j / 64, j % 64);
    for k in q..n {
        let mut w = src.words[k - q] << s;
        if s > 0 && k > q {
            w |= src.words[k - q - 1] >> (64 - s);
        }
        self.words[k] ^= w;
    }

    if j > 0 {
        let (q, s) = ((r - j) / 64, (r - j) % 64);
        for k in 0..n - q {
            let mut w = src.words[k + q] >> s;
            if s > 0 && k + q + 1 < n {
                w |= src.words[k + q + 1] << (64 - s);
            }
            self.words[k] ^= w;
        }
    }

    self.words[n - 1] &= Self::mask(r);
}

pub fn shift(&self, j: usize) -> DenseBitVec {
    let mut res = DenseBitVec::zero(self.r);
    res.xor_shifted(self, j);
    return res;
}

```

```

    }

    pub fn mul(&self, other: &DenseBitVec) -> DenseBitVec {
        assert_eq!(self.r, other.r, "vectors from different rings");
        let mut res = DenseBitVec::zero(self.r);
        for i in self.support() {
            res.xor_shifted(other, i);
        }
        return res;
    }

    pub fn square(&self) -> DenseBitVec {
        let mut res = DenseBitVec::zero(self.r);
        for i in self.support() {
            res.flip(2 * i % self.r);
        }
        return res;
    }

    pub fn inverse(&self) -> Option<DenseBitVec> {
        let r = self.r;
        let len = (r + 1).div_ceil(64);

        let mut r0 = vec![0u64; len];
        r0[0] = 1;
        r0[r / 64] |= 1u64 << (r % 64);
        let mut r1 = self.words.clone();
        r1.resize(len, 0);

        let mut v0 = DenseBitVec::zero(r);
        let mut v1 = DenseBitVec::one(r);

        loop {
            let d1 = degree(&r1)?;
            if d1 == 0 {
                return Some(v1);
            }

            while let Some(d0) = degree(&r0) {
                if d0 < d1 {
                    break;
                }
                xor_shifted_plain(&mut r0, &r1, d0 - d1);
                v0.xor_shifted(&v1, d0 - d1);
            }

            std::mem::swap(&mut r0, &mut r1);
            std::mem::swap(&mut v0, &mut v1);
        }
    }
}

fn degree(words: &[u64]) -> Option<usize> {
    let k = words.iter().rposition(|&w| w != 0)?;
    return Some(k * 64 + 63 - words[k].leading_zeros() as usize);
}

fn xor_shifted_plain(dst: &mut [u64], src: &[u64], s: usize) {
    let (q, s) = (s / 64, s % 64);
    for k in q..dst.len() {
        let mut w = src[k - q] << s;
        if s > 0 && k > q {
            w |= src[k - q - 1] >> (64 - s);
        }
    }
}

```

```

        }
        dst[k] ^= w;
    }
}

impl From<&SparseBitVec> for DenseBitVec {
    fn from(value: &SparseBitVec) -> Self {
        let len = value.r.div_ceil(64);
        let mut words: Vec<u64> = vec![0u64; len];
        for &bit in &value.set_bits {
            let word = bit / 64;
            let bit_idx = bit % 64;

            words[word] |= 1u64 << bit_idx;
        }

        return Self { r: value.r, words };
    }
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub struct SparseBitVec {
    r: usize,
    set_bits: Vec<usize>,
}

impl SparseBitVec {
    pub fn random<R: Rng>(r: usize, d: usize, rng: &mut R) -> Self {
        let mut set_bits = sample(rng, r, d).into_vec();
        set_bits.sort_unstable();

        return Self { r, set_bits };
    }

    pub fn from_positions(r: usize, mut positions: Vec<usize>) -> Self {
        positions.sort_unstable();
        assert!(
            positions.windows(2).all(|p| p[0] < p[1]),
            "positions must be distinct"
        );
        assert!(
            positions.last().is_none_or(|&p| p < r),
            "positions must be less than r = {r}"
        );
        return Self {
            r,
            set_bits: positions,
        };
    }

    pub fn r(&self) -> usize {
        return self.r;
    }

    pub fn weight(&self) -> usize {
        return self.set_bits.len();
    }

    pub fn positions(&self) -> &[usize] {
        return &self.set_bits;
    }

    pub fn mul(&self, other: &DenseBitVec) -> DenseBitVec {

```

```

        assert_eq!(self.r, other.r, "vectors from different rings");
        let mut res = DenseBitVec::zero(self.r);
        for &i in &self.set_bits {
            res.xor_shifted(other, i);
        }
        return res;
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use rand::{SeedableRng, rngs::StdRng};

    const RINGS: [usize; 3] = [11, 587, 1019];
    const WORD_EDGES: [usize; 7] = [5, 11, 63, 64, 65, 128, 587];

    fn rng() -> StdRng {
        return StdRng::seed_from_u64(4);
    }

    fn dense(r: usize, positions: &[usize]) -> DenseBitVec {
        return DenseBitVec::from(&SparseBitVec::from_positions(r,
positions.to_vec()));
    }

    fn monomial(r: usize, j: usize) -> DenseBitVec {
        return dense(r, &[j]);
    }

    fn all_ones(r: usize) -> DenseBitVec {
        return dense(r, &(0..r).collect::<Vec<_>>());
    }

    fn random_dense<R: Rng>(r: usize, rng: &mut R) -> DenseBitVec {
        let bytes: Vec<u8> = (0..r.div_ceil(8))
            .map(|_| rng.gen_range(0..=u8::MAX))
            .collect();
        return DenseBitVec::from_bytes(r, &bytes);
    }

    fn naive_shift(a: &DenseBitVec, j: usize) -> DenseBitVec {
        let r = a.r();
        let mut out = DenseBitVec::zero(r);
        for i in 0..r {
            if a.get(i) {
                out.flip((i + j) % r);
            }
        }
        return out;
    }

    fn naive_mul(a: &DenseBitVec, b: &DenseBitVec) -> DenseBitVec {
        let r = a.r();
        let mut out = DenseBitVec::zero(r);
        for i in 0..r {
            if !a.get(i) {
                continue;
            }
            for j in 0..r {
                if b.get(j) {
                    out.flip((i + j) % r);
                }
            }
        }
    }
}

```

```

    }
    }
    return out;
}

fn fermat_inverse(a: &DenseBitVec) -> DenseBitVec {
    let r = a.r();
    let mut out = DenseBitVec::one(r);
    for _ in 0..r - 2 {
        out = out.square().mul(a);
    }
    return out.square();
}

#[test]
fn zero_and_one() {
    for r in WORD_EDGES {
        let zero = DenseBitVec::zero(r);
        assert_eq!(zero.r(), r);
        assert!(zero.is_zero());
        assert_eq!(zero.weight(), 0);

        let one = DenseBitVec::one(r);
        assert_eq!(one.r(), r);
        assert!(!one.is_zero());
        assert_eq!(one.support(), vec![0]);
    }
}

#[test]
fn bit_string_lists_coefficients_from_a0() {
    assert_eq!(dense(5, &[0, 2]).to_bit_string(), "10100");
    assert_eq!(dense(11, &[1, 4, 5, 6, 8]).to_bit_string(),
"01001110100");
    assert_eq!(DenseBitVec::zero(3).to_bit_string(), "000");
}

#[test]
fn get_set_flip_across_word_boundaries() {
    let r = 587;
    let mut v = DenseBitVec::zero(r);
    let positions = [0, 1, 63, 64, 127, 128, 575, 576, 586];
    for (count, &i) in positions.iter().enumerate() {
        assert!(v.get(i));
        v.set(i, true);
        assert!(v.get(i));
        assert_eq!(v.weight(), count + 1);
    }
    assert_eq!(v.support(), positions.to_vec());

    for &i in &positions {
        v.flip(i);
        assert!(v.get(i));
    }
    assert!(v.is_zero());

    v.set(64, true);
    v.set(64, true);
    assert_eq!(v.weight(), 1);
    v.set(64, false);
    assert!(v.is_zero());
}

```

```

#[test]
#[should_panic]
fn get_beyond_r_panics() {
    DenseBitVec::zero(587).get(587);
}

#[test]
#[should_panic]
fn flip_beyond_r_panics() {
    DenseBitVec::zero(11).flip(11);
}

#[test]
fn from_bytes_uses_lsb_first_order() {
    let v = DenseBitVec::from_bytes(11, &[0b0000_0101, 0b0000_0100]);
    assert_eq!(v.support(), vec![0, 2, 10]);
    assert_eq!(v.to_bytes(), vec![0b0000_0101, 0b0000_0100]);
}

#[test]
fn from_bytes_clears_bits_beyond_r() {
    for r in [11, 63, 64, 65, 587, 12323] {
        let v = DenseBitVec::from_bytes(r, &vec![0xFF; r.div_ceil(8)]);
        assert_eq!(v.weight(), r, "r = {r}");
        assert_eq!(v, all_ones(r), "r = {r}");

        let bytes = v.to_bytes();
        assert_eq!(bytes.len(), r.div_ceil(8));
        if r % 8 != 0 {
            assert_eq!(*bytes.last().unwrap(), (1u8 << (r % 8)) - 1, "r = {r}");
        }
    }
}

#[test]
fn from_bytes_pads_short_input() {
    for r in [11, 587, 12323] {
        let v = DenseBitVec::from_bytes(r, &vec![0xFF; r / 8]);
        assert_eq!(v.r(), r);
        assert_eq!(v.weight(), r / 8 * 8);
        assert_eq!(v.to_bytes().len(), r.div_ceil(8));
        assert_eq!(v.xor(&DenseBitVec::zero(r)), v);
    }
    assert!(DenseBitVec::from_bytes(587, &[]).is_zero());
}

#[test]
#[should_panic]
fn from_bytes_rejects_too_many_bytes() {
    DenseBitVec::from_bytes(587, &[0; 75]);
}

#[test]
fn bytes_roundtrip() {
    let mut rng = rng();
    for r in WORD_EDGES {
        for _ in 0..50 {
            let v = random_dense(r, &mut rng);
            assert_eq!(DenseBitVec::from_bytes(r, &v.to_bytes()), v);
        }
    }
}

```

```

#[test]
fn random_odd_has_invertible_shape() {
    let mut rng = rng();
    for r in [11, 63, 64, 65, 587] {
        for _ in 0..1000 {
            let a = DenseBitVec::random_odd(r, &mut rng);
            assert_eq!(a.r(), r);
            assert_eq!(a.weight() % 2, 1, "r = {r}");
            assert_ne!(a.weight(), r, "r = {r}: all ones is not
invertible");
            assert!(a.support().iter().all(|&i| i < r), "r = {r}: tail is
dirty");
        }
    }
}

#[test]
fn sparse_random_has_exact_weight() {
    let mut rng = rng();
    for (r, d) in [(11, 3), (587, 15), (1019, 21), (12323, 71)] {
        for _ in 0..100 {
            let h = SparseBitVec::random(r, d, &mut rng);
            assert_eq!(h.r(), r);
            assert_eq!(h.weight(), d);
            assert!(h.positions().windows(2).all(|p| p[0] < p[1]));
            assert!(h.positions().iter().all(|&i| i < r));
        }
    }
}

#[test]
fn sparse_from_positions_sorts() {
    let h = SparseBitVec::from_positions(11, vec![8, 0, 3]);
    assert_eq!(h.positions(), &[0, 3, 8]);
    assert_eq!(h.weight(), 3);
}

#[test]
#[should_panic]
fn sparse_from_positions_rejects_duplicates() {
    SparseBitVec::from_positions(11, vec![1, 5, 1]);
}

#[test]
#[should_panic]
fn sparse_from_positions_rejects_out_of_range() {
    SparseBitVec::from_positions(11, vec![0, 11]);
}

#[test]
fn dense_from_sparse_keeps_positions() {
    let mut rng = rng();
    for r in WORD_EDGES {
        let h = SparseBitVec::random(r, r.min(15), &mut rng);
        let d = DenseBitVec::from(&h);
        assert_eq!(d.r(), r);
        assert_eq!(d.support(), h.positions().to_vec());
    }
}

#[test]
fn support_matches_get() {

```

```

        let mut rng = rng();
        for r in WORD_EDGES {
            let v = random_dense(r, &mut rng);
            let expected: Vec<usize> = (0..r).filter(|&i|
v.get(i)).collect();
            assert_eq!(v.support(), expected);
            assert_eq!(v.weight(), expected.len());
        }
    }

#[test]
fn xor_is_bitwise_addition() {
    let mut rng = rng();
    for r in WORD_EDGES {
        let a = random_dense(r, &mut rng);
        let b = random_dense(r, &mut rng);
        let c = a.xor(&b);
        for i in 0..r {
            assert_eq!(c.get(i), a.get(i) != b.get(i));
        }
        assert_eq!(c, b.xor(&a));
        assert_eq!(a.xor(&a), DenseBitVec::zero(r));
        assert_eq!(a.xor(&DenseBitVec::zero(r)), a);

        let mut d = a.clone();
        d.xor_assign(&b);
        assert_eq!(d, c);
    }
}

#[test]
fn xor_sparse_matches_dense_xor() {
    let mut rng = rng();
    for r in WORD_EDGES {
        let a = random_dense(r, &mut rng);
        let e = SparseBitVec::random(r, r.min(10), &mut rng);
        let mut b = a.clone();
        b.xor_sparse_assign(&e);
        assert_eq!(b, a.xor(&DenseBitVec::from(&e)));
    }
}

#[test]
#[should_panic]
fn xor_rejects_different_r() {
    DenseBitVec::zero(587).xor(&DenseBitVec::zero(1019));
}

#[test]
fn shift_small_examples() {
    assert_eq!(dense(5, &[1, 4]).shift(2), dense(5, &[1, 3]));
    assert_eq!(monomial(11, 10).shift(1), DenseBitVec::one(11));
    assert_eq!(monomial(587, 586).shift(1), DenseBitVec::one(587));
    assert_eq!(monomial(64, 63).shift(1), DenseBitVec::one(64));
}

#[test]
fn shift_of_monomials() {
    for r in WORD_EDGES {
        let ks = [
            0,
            1,
            2,

```

```

        31,
        63,
        64,
        65,
        127,
        128,
        r - 1,
        r,
        r + 1,
        2 * r + 3,
    ];
    for j in 0..r {
        for &k in &ks {
            assert_eq!(
                monomial(r, j).shift(k),
                monomial(r, (j + k) % r),
                "r = {r}, x^{j} * x^{k}"
            );
        }
    }
}

#[test]
fn shift_matches_naive() {
    let mut rng = rng();
    for r in WORD_EDGES {
        let a = random_dense(r, &mut rng);
        for j in 0..r {
            let shifted = a.shift(j);
            assert_eq!(shifted, naive_shift(&a, j), "r = {r}, j = {j}");
            assert_eq!(shifted.weight(), a.weight());
        }
    }
}

#[test]
fn shift_by_r_is_identity_and_shifts_compose() {
    let mut rng = rng();
    for r in WORD_EDGES {
        let a = random_dense(r, &mut rng);
        assert_eq!(a.shift(0), a);
        assert_eq!(a.shift(r), a);
        for _ in 0..50 {
            let j = rng.gen_range(0..r);
            let k = rng.gen_range(0..r);
            assert_eq!(a.shift(j).shift(k), a.shift((j + k) % r));
        }
    }
}

#[test]
fn convolution_small_example() {
    let a = SparseBitVec::from_positions(5, vec![0, 2]);
    let b = dense(5, &[1, 4]);
    let expected = dense(5, &[3, 4]);
    assert_eq!(a.mul(&b), expected);
    assert_eq!(DenseBitVec::from(&a).mul(&b), expected);
    assert_eq!(b.mul(&DenseBitVec::from(&a)), expected);
}

#[test]
fn multiplying_by_monomial_is_shift() {

```

```

    let mut rng = rng();
    for r in WORD_EDGES {
        let a = random_dense(r, &mut rng);
        for j in [0, 1, 63, 64, r - 1].into_iter().filter(|&j| j < r) {
            assert_eq!(a.mul(&monomial(r, j)), a.shift(j));
            assert_eq!(SparseBitVec::from_positions(r, vec![j]).mul(&a),
a.shift(j));
        }
    }

}

#[test]
fn sparse_mul_matches_naive() {
    let mut rng = rng();
    for (r, d) in [(11, 3), (587, 15), (1019, 21)] {
        for _ in 0..10 {
            let h = SparseBitVec::random(r, d, &mut rng);
            let b = random_dense(r, &mut rng);
            assert_eq!(h.mul(&b), naive_mul(&DenseBitVec::from(&h), &b),
"r = {r}");
        }
    }
}

#[test]
fn dense_mul_matches_naive() {
    let mut rng = rng();
    for r in WORD_EDGES {
        for _ in 0..3 {
            let a = random_dense(r, &mut rng);
            let b = random_dense(r, &mut rng);
            assert_eq!(a.mul(&b), naive_mul(&a, &b), "r = {r}");
        }
    }
}

#[test]
fn sparse_mul_matches_dense_mul() {
    let mut rng = rng();
    for (r, d) in [(587, 15), (12323, 71)] {
        let h = SparseBitVec::random(r, d, &mut rng);
        let b = random_dense(r, &mut rng);
        assert_eq!(h.mul(&b), DenseBitVec::from(&h).mul(&b), "r = {r}");
    }
}

#[test]
fn ring_laws() {
    let mut rng = rng();
    for r in RINGS {
        let a = random_dense(r, &mut rng);
        let b = random_dense(r, &mut rng);
        let c = random_dense(r, &mut rng);
        let zero = DenseBitVec::zero(r);
        let one = DenseBitVec::one(r);

        assert_eq!(a.mul(&b), b.mul(&a), "commutative, r = {r}");
        assert_eq!(a.mul(&one), a, "identity, r = {r}");
        assert_eq!(a.mul(&zero), zero, "zero, r = {r}");
        assert_eq!(
            a.mul(&b.xor(&c)),
            a.mul(&b).xor(&a.mul(&c)),
            "distributive, r = {r}"
        );
    }
}

```

```

    );
    assert_eq!(a.mul(&b).mul(&c), a.mul(&b.mul(&c)), "associative, r
= {r}");
    }
}

#[test]
fn ring_has_zero_divisors() {
    for r in RINGS {
        assert!(dense(r, &[0, 1]).mul(&all_ones(r)).is_zero(), "r =
{r}");
    }
}

#[test]
#[should_panic]
fn mul_rejects_different_r() {
    DenseBitVec::one(587).mul(&DenseBitVec::one(1019));
}

#[test]
fn square_doubles_exponents() {
    let mut rng = rng();
    for (r, d) in [(11, 3), (587, 15), (1019, 21)] {
        let h = SparseBitVec::random(r, d, &mut rng);
        let mut expected: Vec<usize> = h.positions().iter().map(|&i| 2 *
i % r).collect();
        expected.sort_unstable();
        let squared = DenseBitVec::from(&h).square();
        assert_eq!(squared.support(), expected, "r = {r}");
    }
}

#[test]
fn square_matches_mul() {
    let mut rng = rng();
    for r in [5, 11, 63, 65, 587] {
        let a = random_dense(r, &mut rng);
        assert_eq!(a.square(), a.mul(&a), "r = {r}");
    }
}

#[test]
fn inverse_of_random_odd() {
    let mut rng = rng();
    for r in RINGS {
        for _ in 0..20 {
            let a = DenseBitVec::random_odd(r, &mut rng);
            let inv = a.inverse().expect("odd weight, not all ones");
            assert_eq!(a.mul(&inv), DenseBitVec::one(r), "r = {r}");
        }
    }
}

#[test]
fn inverse_of_sparse_key() {
    let mut rng = rng();
    for (r, d) in [(11, 3), (587, 15), (1019, 21)] {
        for _ in 0..20 {
            let h = DenseBitVec::from(&SparseBitVec::random(r, d, &mut
rng));
            let inv = h.inverse().expect("odd d");
            assert_eq!(h.mul(&inv), DenseBitVec::one(r), "r = {r}");

```

```

    }
  }
}

#[test]
fn inverse_of_one_is_one() {
  for r in RINGS {
    assert_eq!(DenseBitVec::one(r).inverse(),
Some(DenseBitVec::one(r)));
  }
}

#[test]
fn non_invertible_elements() {
  let mut rng = rng();
  for r in RINGS {
    assert_eq!(DenseBitVec::zero(r).inverse(), None, "zero, r =
{r}");
    assert_eq!(all_ones(r).inverse(), None, "all ones, r = {r}");
    assert_eq!(dense(r, &[0, 1]).inverse(), None, "1 + x, r = {r}");
    for _ in 0..20 {
      let mut even = DenseBitVec::random_odd(r, &mut rng);
      even.flip(0);
      assert_eq!(even.inverse(), None, "even weight, r = {r}");
    }
  }
}

#[test]
fn inverse_matches_fermat() {
  let mut rng = rng();
  for r in [11, 13, 59, 587] {
    for _ in 0..5 {
      let a = DenseBitVec::random_odd(r, &mut rng);
      assert_eq!(a.inverse(), Some(fermat_inverse(&a)), "r = {r}");
    }
  }
}

#[test]
fn levell_ring() {
  let mut rng = rng();
  let r = 12323;
  let h0 = SparseBitVec::random(r, 71, &mut rng);
  let h1 = DenseBitVec::from(&SparseBitVec::random(r, 71, &mut rng));
  let q = h0.mul(&h1.inverse().unwrap());
  assert_eq!(h1.mul(&q), DenseBitVec::from(&h0));

  let a = DenseBitVec::random_odd(r, &mut rng);
  assert_eq!(a.mul(&a.inverse().unwrap()), DenseBitVec::one(r));
}

#[test]
fn spec_mini_example() {
  let r = 11;
  let h0 = SparseBitVec::from_positions(r, vec![0, 1, 2]);
  let h1 = SparseBitVec::from_positions(r, vec![0, 1, 3]);
  let h1_dense = DenseBitVec::from(&h1);

  let h1_inv = h1_dense.inverse().unwrap();
  assert_eq!(h1_inv, dense(r, &[0, 1, 3, 6, 7, 8, 10]));
  assert_eq!(h1.mul(&h1_inv), DenseBitVec::one(r));
}

```

```

        let q = h0.mul(&h1_inv);
        assert_eq!(q, dense(r, &[1, 4, 5, 6, 8]));
        assert_eq!(h1.mul(&q), DenseBitVec::from(&h0));

        assert_eq!(h1_dense.square(), dense(r, &[0, 2, 6]));

        let m = dense(r, &[0, 2, 3, 8]);
        let mq = m.mul(&q);
        assert_eq!(mq, dense(r, &[0, 2, 8, 10]));

        let c0 = m.clone();
        let mut c1 = mq;
        c1.xor_sparse_assign(&SparseBitVec::from_positions(r, vec![5]));
        assert_eq!(c1, dense(r, &[0, 2, 5, 8, 10]));

        let s = h0.mul(&c0).xor(&h1.mul(&c1));
        assert_eq!(s, dense(r, &[5, 6, 8]));
        assert_eq!(s, h1_dense.shift(5));
    }
}

```

Листинг А.2 – Реализация криптосистемы Мак-Элиса на кодах QC-MDPC

```

use super::bit_vecs::{DenseBitVec, SparseBitVec};
use rand::{
    Rng, SeedableRng,
    rngs::{OsRng, StdRng},
    seq::index::sample,
};

pub struct QcMdpParams {
    r: usize,
    d: usize,
    t: usize,
}

pub const CRYPTO_PARAMS: QcMdpParams = QcMdpParams {
    r: 12323,
    d: 71,
    t: 134,
};

const MAX_ITERATIONS: usize = 10;

#[derive(Clone)]
pub struct PublicKey {
    r: usize,
    t: usize,
    a: DenseBitVec,
    b: DenseBitVec,
}

impl QcMdpParams {
    pub fn r(&self) -> usize {
        return self.r;
    }

    pub fn d(&self) -> usize {
        return self.d;
    }

    pub fn t(&self) -> usize {
        return self.t;
    }
}

```

```

    }
}

impl PublicKey {
    pub fn r(&self) -> usize {
        return self.r;
    }

    pub fn t(&self) -> usize {
        return self.t;
    }

    pub fn a(&self) -> &DenseBitVec {
        return &self.a;
    }

    pub fn b(&self) -> &DenseBitVec {
        return &self.b;
    }
}

pub struct PrivateKey {
    h0: SparseBitVec,
    h1: SparseBitVec,
    a_inv: DenseBitVec,
    public: PublicKey,
}

impl PrivateKey {
    pub fn h0(&self) -> &SparseBitVec {
        return &self.h0;
    }

    pub fn h1(&self) -> &SparseBitVec {
        return &self.h1;
    }

    pub fn a_inv(&self) -> &DenseBitVec {
        return &self.a_inv;
    }
}

pub struct QcMdp {
}

impl QcMdp {
    pub fn keygen(params: QcMdpParams) -> (PrivateKey, PublicKey) {
        let mut rng = StdRng::from_rng(&mut OsRng).unwrap();

        let h0 = SparseBitVec::random(params.r, params.d, &mut rng);
        let h1 = SparseBitVec::random(params.r, params.d, &mut rng);
        let h1_inv = DenseBitVec::from(&h1)
            .inverse()
            .expect("h1 of odd weight d is invertible");
        let q = h0.mul(&h1_inv);

        let a = DenseBitVec::random_odd(params.r, &mut rng);
        let a_inv = a
            .inverse()
            .expect("a of odd weight other than all ones is invertible");
        let b = a.mul(&q);

        let pub_key = PublicKey {
            r: params.r,

```

```

        t: params.t,
        a,
        b,
    };

    return (
        PrivateKey {
            h0,
            h1,
            a_inv,
            public: pub_key.clone(),
        },
        pub_key,
    );
}

fn encrypt_block<R: Rng>(
    block: &[u8],
    key: &PublicKey,
    rng: &mut R,
) -> (DenseBitVec, DenseBitVec) {
    let m = DenseBitVec::from_bytes(key.r, block);

    let (low, high): (Vec<usize>, Vec<usize>) = sample(rng, 2 * key.r,
key.t)
        .into_iter()
        .partition(|&p| p < key.r);
    let e0 = SparseBitVec::from_positions(key.r, low);
    let e1 = SparseBitVec::from_positions(key.r, high.into_iter().map(|p|
p - key.r).collect());

    let mut c0 = m.mul(&key.a);
    c0.xor_sparse_assign(&e0);

    let mut c1 = m.mul(&key.b);
    c1.xor_sparse_assign(&e1);

    return (c0, c1);
}

pub fn encrypt(plain_bytes: &[u8], key: &PublicKey) -> Vec<u8> {
    let mut rng = StdRng::from_rng(&mut OsRng).unwrap();

    let k = key.r / 8;
    let mut bytes = Vec::from(plain_bytes);
    let mut result = vec![];

    bytes.push(1);

    let padding = (k - (bytes.len() % k)) % k;
    bytes.resize(bytes.len() + padding, 0);

    for block in bytes.chunks(k) {
        let (c0, c1) = Self::encrypt_block(block, key, &mut rng);

        result.extend_from_slice(&c0.to_bytes());
        result.extend_from_slice(&c1.to_bytes());
    }

    return result;
}

fn counters(s: &DenseBitVec, h: &SparseBitVec) -> Vec<usize> {

```

```

    let r = s.r();
    return (0..r)
        .map(|j| {
            h.positions()
                .iter()
                .filter(|&p| s.get((j + p) % r))
                .count()
        })
        .collect();
}

fn threshold(s: &DenseBitVec) -> usize {
    return ((0.0069722 * s.weight() as f64 + 13.530).floor() as
usize).max(36);
}

fn decode(s: &mut DenseBitVec, key: &PrivateKey) -> Option<(DenseBitVec,
DenseBitVec)> {
    let r = key.public.r;
    let t = key.public.t;
    let h0 = &key.h0;
    let h1 = &key.h1;

    let mut e0 = DenseBitVec::zero(r);
    let mut e1 = DenseBitVec::zero(r);

    for _ in 0..MAX_ITERATIONS {
        if s.is_zero() {
            break;
        }

        let mut flipped = false;

        let upc0 = Self::counters(&s, h0);
        let upc1 = Self::counters(&s, h1);
        let t_flip = Self::threshold(&s);

        for j in 0..r {
            if upc0[j] >= t_flip {
                e0.flip(j);
                flipped = true;
                for &p in h0.positions() {
                    s.flip((j + p) % r);
                }
            }
            if upc1[j] >= t_flip {
                e1.flip(j);
                flipped = true;
                for &p in h1.positions() {
                    s.flip((j + p) % r);
                }
            }
        }

        if !flipped {
            return None;
        }

        if !s.is_zero() || e0.weight() + e1.weight() != t {
            return None;
        }
    }
}

```

```

        return Some((e0, e1));
    }

    pub fn decrypt(encrypted_bytes: &[u8], key: &PrivateKey) ->
Option<Vec<u8>> {
    let r = key.public.r;

    let k = r / 8;
    let mut result = vec![];

    let half = r.div_ceil(8);
    if encrypted_bytes.len() % (2 * half) != 0 {
        return None;
    }

    for (c0_bytes, c1_bytes) in encrypted_bytes
        .chunks(2 * half)
        .map(|block| block.split_at(half))
    {
        let mut c0 = DenseBitVec::from_bytes(r, c0_bytes);
        let mut c1 = DenseBitVec::from_bytes(r, c1_bytes);
        let mut s = key.h0.mul(&c0).xor(&key.h1.mul(&c1));

        let (e0, e1) = Self::decode(&mut s, key)?;

        c0.xor_assign(&e0);
        let m = c0.mul(&key.a_inv);

        c1.xor_assign(&e1);
        if c1 != m.mul(&key.public.b) {
            return None;
        }

        result.extend_from_slice(&m.to_bytes()[..k]);
    }

    while result.last() == Some(&0) {
        result.pop();
    }
    if result.pop() != Some(1) {
        return None;
    }

    return Some(result);
}

}

#[cfg(test)]
mod tests {
    use super::*;

    fn rng() -> StdRng {
        return StdRng::seed_from_u64(4);
    }

    fn random_bytes<R: Rng>(len: usize, rng: &mut R) -> Vec<u8> {
        return (0..len).map(|_| rng.gen_range(0..=u8::MAX)).collect();
    }

    #[test]
    fn keys_satisfy_h1_q_eq_h0() {
        let (private, public) = QcMdp::keygen(CRYPTO_PARAMS);
        // B = a q, поэтому q = a-1 B

```

```

    let q = private.a_inv.mul(&public.b);
    assert_eq!(private.h1.mul(&q), DenseBitVec::from(&private.h0));
    assert_eq!(public.a.mul(&private.a_inv), DenseBitVec::one(public.r));
}

#[test]
fn syndrome_depends_only_on_errors() {
    let (private, public) = QcMdp::keygen(CRYPTO_PARAMS);
    let mut rng = rng();
    let block = random_bytes(public.r / 8, &mut rng);
    let m = DenseBitVec::from_bytes(public.r, &block);

    let (c0, c1) = QcMdp::encrypt_block(&block, &public, &mut rng);
    let e0 = c0.xor(&m.mul(&public.a));
    let e1 = c1.xor(&m.mul(&public.b));
    assert_eq!(e0.weight() + e1.weight(), public.t);

    let s = private.h0.mul(&c0).xor(&private.h1.mul(&c1));
    let expected = e0
        .mul(&DenseBitVec::from(&private.h0))
        .xor(&e1.mul(&DenseBitVec::from(&private.h1)));
    assert_eq!(s, expected);
}

#[test]
fn roundtrip_messages() {
    let (private, public) = QcMdp::keygen(CRYPTO_PARAMS);
    let mut rng = rng();
    let k = public.r / 8;
    let half = public.r.div_ceil(8);

    let mut messages = vec![
        vec![],
        b"a".to_vec(),
        "Шифр McEliece на кодах QC-MDPC".as_bytes().to_vec(),
        vec![0; 10],
        vec![1; 5],
    ];

    for len in [k - 1, k, k + 1, 2 * k, 3 * k + 17] {
        messages.push(random_bytes(len, &mut rng));
    }

    for message in &messages {
        let encrypted = QcMdp::encrypt(message, &public);
        // дополнение всегда есть: len + 1 байт, округлённые вверх до
        // блоков по k
        let blocks = (message.len() + 1).div_ceil(k);
        assert_eq!(encrypted.len(), blocks * 2 * half, "len = {}",
        message.len());
        assert_eq!(
            QcMdp::decrypt(&encrypted, &private).as_ref(),
            Some(message),
            "len = {}",
            message.len()
        );
    }
}

#[test]
fn many_blocks_without_failures() {
    let (private, public) = QcMdp::keygen(CRYPTO_PARAMS);
    let mut rng = rng();
    let message = random_bytes(50 * (public.r / 8) - 1, &mut rng);

```

```

        let encrypted = QcMdpс::encrypt(&message, &public);
        assert_eq!(QcMdpс::decrypt(&encrypted, &private), Some(message));
    }

    #[test]
    fn rejects_malformed_ciphertext() {
        let (private, public) = QcMdpс::keygen(CRYPTO_PARAMS);
        assert_eq!(QcMdpс::decrypt(&[], &private), None);

        let mut truncated = QcMdpс::encrypt(b"x", &public);
        truncated.pop();
        assert_eq!(QcMdpс::decrypt(&truncated, &private), None);

        // лишняя ошибка: t + 1 единиц, проверка веса должна отказать
        let mut tampered = QcMdpс::encrypt(b"x", &public);
        tampered[0] ^= 1;
        assert_eq!(QcMdpс::decrypt(&tampered, &private), None);
    }

    #[test]
    fn wrong_key_does_not_decrypt() {
        let (_, public) = QcMdpс::keygen(CRYPTO_PARAMS);
        let (other_private, _) = QcMdpс::keygen(CRYPTO_PARAMS);
        let encrypted = QcMdpс::encrypt(b"secret", &public);
        assert_eq!(QcMdpс::decrypt(&encrypted, &other_private), None);
    }
}

```

ПРИЛОЖЕНИЕ Б **(обязательное)** **Блок-схема алгоритма**

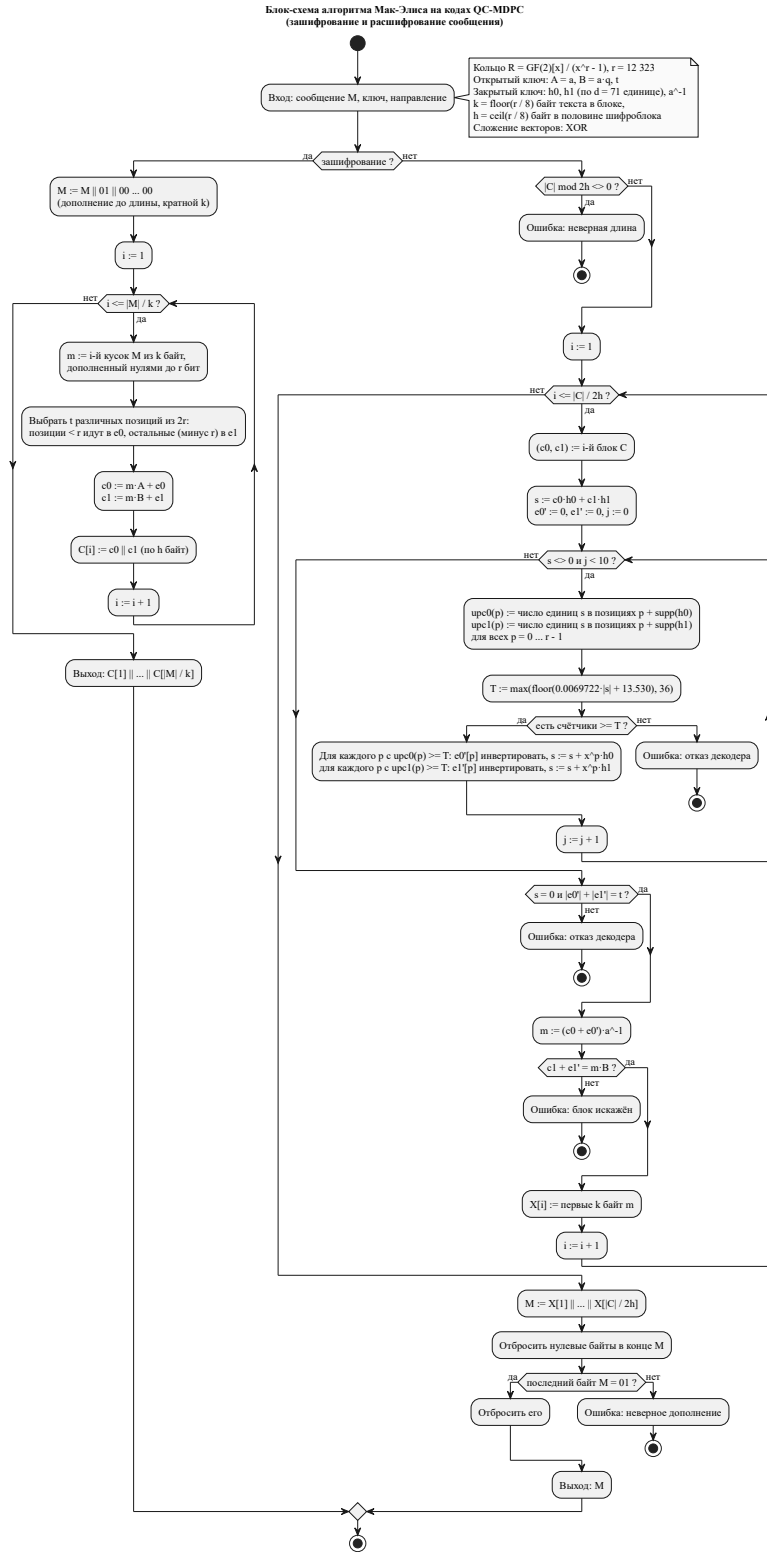


Рисунок Б.1 – Блок-схема алгоритма